

RUANA

Red Unida de Apoyo entre Negocios Aliados

Sistema de control, coordinación y reputación profesional para redes locales de profesionales y pequeños negocios. RUANA no es una red social ni un marketplace: aplica reglas operativas, score, estados y trazabilidad sobre el comportamiento de los aliados.

Principio rector: El panel no piensa. El motor decide. El panel solo refleja estado.

Tabla de contenidos

- [Estado del proyecto](#)
- [Arquitectura](#)
- [Stack tecnológico](#)
- [Estructura del repositorio](#)
- [Requisitos](#)
- [Configuración de entorno](#)
- [Ejecución local](#)
- [Base de datos](#)
- [Autenticación y sesiones](#)
- [Funcionalidades implementadas](#)
- [Flujo general de uso](#)
- [API \(resumen\)](#)
- [Scripts disponibles](#)
- [Pruebas](#)
- [Despliegue](#)
- [Pendientes y roadmap](#)
- [Documentación adicional](#)
- [Límites conocidos](#)
- [Licencia](#)

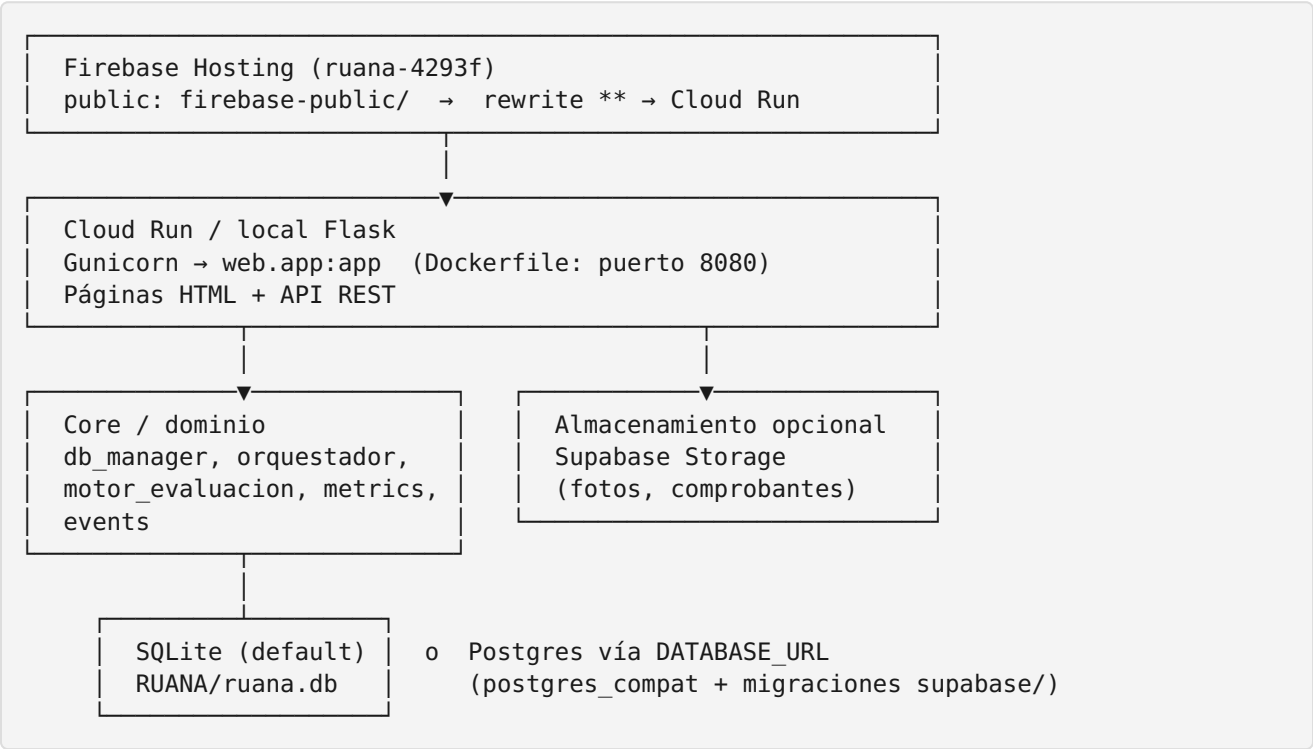
Estado del proyecto

Campo	Valor deducido del código / docs del repo
Fase	Pre-MVP avanzada (ROADMAP.md)
Hito activo	Hito 2 — cierre de superficie crítica de seguridad y permisos
Backend	Flask (RUANA/web/app.py)
Persistencia	SQLite local • Postgres/Supabase si DATABASE_URL está definida

Campo	Valor deducido del código / docs del repo
Hosting previsto	Firebase Hosting → rewrite a Cloud Run (<code>firebase.json</code>)
Versión documentada en <code>RUANA/README.md</code>	0.9 (pre-MVP) — esa documentación interna está parcialmente desactualizada respecto al código

No se ha encontrado un archivo `LICENSE` ni un número de versión semántica en `package.json` / `pyproject.toml` más allá de lo indicado en la documentación interna.

Arquitectura



Capas principales:

Capa	Ubicación	Rol
Frontend	<code>RUANA/web/*.html</code> + <code>static/</code>	Presentación y captura de acciones
Backend HTTP	<code>RUANA/web/app.py</code>	Rutas, validación, orquestación HTTP, auth
Dominio / BD	<code>RUANA/core/db_manager.py</code>	Persistencia, reglas de negocio, score, grupos, contactos
Motor	<code>RUANA/engines/motor_evaluacion.py</code>	Evaluación periódica (verde/amarillo/rojo)
Infra	<code>Dockerfile</code> , <code>firebase.json</code> , <code>.github/workflows/</code> , <code>scripts/</code>	Build, deploy, secretos GCP

Stack tecnológico

Backend (Python)

Definido en `RUANA/web/requirements.txt`:

Paquete	Uso
Flask 2.3.3	Servidor web / API
Flask-Cors 4.0.0	CORS (configuración por defecto, sin orígenes restringidos en código)
Werkzeug 2.3.7	Utilidades Flask / uploads
PyJWT 2.8.0	Tokens de sesión / JWT admin
gunicorn	Servidor WSGI en contenedor
python-dotenv	Carga de <code>.env</code> / <code>.env.local</code>
psycopg[binary,pool]	Cliente Postgres
supabase 2.16.0	Cliente Storage / admin Supabase
Pillow	Optimización de fotos de perfil

Desarrollo: `RUANA/web/requirements-dev.txt` añade `pytest`.

Frontend

- HTML + CSS + JavaScript vanilla (sin framework SPA en el flujo principal).
- Páginas principales: `index.html`, `invite.html`, `register.html`, `aliado.html`, `admin.html`.
- Estáticos en `RUANA/web/static/` (`styles.css`, `panel-premium.css`, `ruana-ui.js`, etc.).
- Lucide Icons vía CDN en el panel aliado.

Infraestructura y tooling (raíz del monorepo)

Tecnología	Evidencia
Node.js (tooling)	<code>package.json</code> — Playwright, firebase-tools, CLI supabase
Firebase Hosting	<code>firebase.json</code> , <code>.firebaserc</code> (proyecto <code>ruana-4293f</code>)
Google Cloud Run	<code>Dockerfile</code> , workflows, scripts PowerShell
Supabase (Postgres + Storage)	<code>supabase/migrations/</code> , <code>core/supabase_client.py</code>
Playwright	<code>playwright.config.js</code> , <code>e2e/</code>
GitHub Actions	<code>.github/workflows/</code>

Estructura del repositorio

```
.
├── README.md           # Este documento
├── ROADMAP.md          # Roadmap operativo por hitos
├── package.json        # Scripts npm (deploy, QA, Supabase CLI)
├── Dockerfile          # Imagen Cloud Run (Python 3.13 + gunicorn)
├── firebase.json       # Hosting → rewrite a Cloud Run service "ruana"
├── .env.example        # Plantilla de variables de entorno
├── playwright.config.js
├── e2e/                # Tests E2E Playwright
├── docs/               # Planes QA, specs y planes de hitos
├── scripts/            # Deploy / secretos GCP (PowerShell)
├── supabase/migrations/ # Migraciones Postgres
├── firebase-public/    # Carpeta pública Firebase (solo .gitkeep; el tráfico se
reescribe a Cloud Run)
└── RUANA/              # Aplicación
    ├── README.md       # Documentación de dominio (parcialmente desactualizada)
    ├── config/         # admin_codes, oficios, reglas
    ├── core/           # DB, settings, storage, orquestador
    ├── engines/        # Motor de evaluación
    ├── events/         # Event bus → logs JSONL
    ├── metrics/        # Colector de métricas
    ├── scripts/        # Seed, purga, verify_supabase
    ├── tests/          # Suite pytest
    ├── docs/           # Auth, chat, registro
    ├── utils/
    └── web/            # Flask + HTML/CSS/JS
```

Requisitos

- **Python 3.8+** recomendado para desarrollo local; la imagen Docker usa **Python 3.13-slim**.
- **pip** para dependencias Python.
- **Node.js + npm** solo si vas a ejecutar Playwright, Firebase CLI o scripts npm de la raíz.
- Opcional: cuenta/proyecto **Supabase, Firebase/GCP** para despliegue completo.
- Los scripts de deploy en `scripts/*.ps1` están escritos para **PowerShell** (entorno Windows/local documentado en el repo); en Linux/macOS conviene usar los workflows de GitHub Actions o adaptar los comandos `gcloud / firebase` equivalentes.

Configuración de entorno

Copia la plantilla y completa los valores:

```
cp .env.example .env.local
```

`core/settings.py` carga, en este orden (sin sobrescribir si ya existen): `.env.local` y `.env` desde la **raíz del repositorio**.

Variables documentadas en `.env.example`

Variable	Descripción
<code>FLASK_SECRET_KEY</code>	Secreto Flask / firma JWT. Default de desarrollo: <code>ruana_secret_key_dev</code> (no usar en producción)
<code>RUANA_ADMIN_SESSION_EXPIRES</code>	Segundos de vigencia de sesión admin (default <code>3600</code>)
<code>RUANA_ALIADO_SESSION_EXPIRES</code>	Segundos de vigencia de sesión aliado (default <code>3600</code>)
<code>FIREBASE_PROJECT_ID</code>	Proyecto Firebase/GCP (default <code>ruana-4293f</code>)
<code>GOOGLE_CLOUD_REGION</code>	Región (default <code>europa-west1</code>)
<code>ARTIFACT_REGISTRY_REPOSITORY</code>	Repo Artifact Registry (default <code>ruana</code> en scripts)
<code>SUPABASE_URL</code>	URL del proyecto Supabase
<code>SUPABASE_ANON_KEY</code>	Clave anónima pública
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Clave service role (solo backend)
<code>DATABASE_URL</code>	URL Postgres (pooler recomendado para Cloud Run). Si está vacía → SQLite
<code>RUANA_DB_PATH</code>	Ruta del fichero SQLite (default <code>RUANA/ruana.db</code>)

Variables usadas en código pero no listadas en `.env.example`

Variable	Uso
<code>RUANA_PUBLIC_APP_URL</code> / <code>PUBLIC_APP_URL</code>	URL pública de la app (invitaciones); fallback <code>https://{FIREBASE_PROJECT_ID}.web.app</code>
<code>RUANA_BASE_URL</code>	Base URL para Playwright
<code>RUANA_SKIP_WEBSERVER</code>	Omite arranque automático del servidor en E2E
<code>RUANA_QA_ADMIN_CODE</code>	Código admin en fixtures E2E (default <code>ADMIN001</code>)
<code>RUANA_QA_VIDEO_PAUSE_MS</code> / <code>RUANA_QA_ACTION_PAUSE_MS</code>	Pausas visuales en narrador QA

Ejecución local

1. Dependencias Python

```
cd RUANA/web
pip install -r requirements.txt
# opcional, para tests:
pip install -r requirements-dev.txt
```

2. Arrancar el servidor

Desde `RUANA/web` :

```
python run.py
```

O desde la raíz (ajustando el path de imports según el entorno):

```
python RUANA/web/run.py
```

- Host/puerto con `run.py`: `http://127.0.0.1:5000` (`debug=False` , sin reloader).
- Si se ejecuta `app.py` directamente: `0.0.0.0:5000` con `debug=True` .

Health check: `GET /api/health` → `{"status": "healthy", ...}` (forma exacta de la respuesta: ver implementación en `app.py`).

3. Datos semilla (opcional)

```
cd RUANA
python scripts/seed_aliados.py
```

Inserta de forma idempotente aliados de demo con códigos `ALFA01` , `BETA02` , `GAMA03` , `DELTA04` .

4. Tooling npm (raíz)

```
npm ci
```

Base de datos

Selección del backend

En `RUANA/core/db_manager.py` / `settings.py` :

- Si `DATABASE_URL` está definida → backend **Postgres** (vía `postgres_compat.py` , puente de compatibilidad SQLite→Postgres).
- Si no → **SQLite** en `RUANA_DB_PATH` o `RUANA/ruana.db` .

Tablas principales (SQLite / dominio)

Entre otras: `aliados` , `grupos` , `solicitudes` , `contactos_ruana` , `chat_mensajes` , `score_movimientos` , `evaluaciones` , `evaluaciones_historico` , `invitaciones` , `invitacion_campanas` , `referidos` , `invitaciones_oficio` , `competencia` , `notificaciones_aliado` , `payment_conflicts` , `ingresos_ruana` , `eventos_sistema` , `audit_log` , `migraciones` .

Migraciones Supabase

En `supabase/migrations/` :

Migración	Contenido
<code>20260519000100_init_ruana_clean.sql</code>	Esquema inicial, RLS, buckets Storage, realtime
<code>20260519000200_sqlite_compat_names.sql</code>	Renombres de columnas para compatibilidad

Migración	Contenido
20260519000300_sqlite_compat_types.sql	Ajuste de tipos (bool→int, jsonb→text, etc.)
20260714000100_aliados_foto_perfil_url.sql	Columna foto_perfil_url
20260722000100_aliados_invitado_por_linaje.sql	Linaje invitado_por_codigo / invitado_origen

Buckets esperados (según migraciones / código): ruana-public , ruana-comprobantes , ruana-conflictos .

Aplicar migraciones (requiere CLI y proyecto enlazado):

```
npm run supabase:login
npm run supabase:link
npm run supabase:push
```

Verificación: npm run verify:supabase → RUANA/scripts/verify_supabase.py .

Configuración de dominio

Archivo	Contenido
RUANA/config/oficios_ruana.json	Catálogo oficial (39 oficios principales con especializaciones)
RUANA/config/ruana_reglas_v1.json	Umbral de competencia (35), duración (30 días), purga, apoyo_pct (12.0), deltas del motor, métodos de pago
RUANA/config/admin_codes.json	Códigos de administrador y permisos (leer , escribir , eliminar , configurar)

Autenticación y sesiones

Implementación actual en RUANA/web/app.py (detalle extendido en RUANA/docs/AUTENTICACION_SESIONES_SEGURAS.md ; el documento describe un session_id opaco, pero **el código actual firma un JWT** como session_id).

Aliado

1. POST /api/aliado/login con { "codigo": "..." } .
2. Respuesta con session_id .
3. El frontend guarda el id en sessionStorage y lo envía en la cabecera X-Ruana-Session-Id .
4. Validación: GET /api/aliado/sesion . Logout: POST /api/aliado/logout .
5. Decorador @require_aliado en rutas protegidas.

Admin

1. POST /api/admin/validar con código de config/admin_codes.json .
2. Devuelve session_id , token (JWT) y permisos .
3. El panel admin usa X-Ruana-Session-Id (también se acepta Authorization: Bearer <token>) .

4. Middleware: casi todo `/api/admin/*` exige auth excepto `/api/admin/validar` y `/api/admin/logout`.
5. Escrituras sensibles: `@require_admin_escritura` (permisos `escribir` o `configurar`).
6. El parámetro `?bypass=` en `/admin` **no** concede acceso.

Notas de seguridad

- Store de sesiones / revocación en memoria del proceso (`_RUANA_SESSION_STORE` , `_RUANA_SESSION_REVOKED`): con varios workers o reinicios, la revocación no es compartida; un JWT no expirado puede seguir siendo válido en otra instancia.
- CORS se habilita de forma abierta (`CORS(app)` sin lista de orígenes).
- No usar el secret key por defecto en producción.

Funcionalidades implementadas

Basado en código y tests existentes (no en planes futuros):

Área	Qué hay
Registro de aliados	Invitación → registro → código numérico de 5 dígitos; validación email/teléfono; catálogo de oficios
Asignación de grupos	Por código postal y oficio/especialización; máx. 5 grupos/CP; fusión/viabilidad
Panel aliado	Perfil, score/estado, solicitudes, directorio del grupo, contactos, chat, notificaciones, pagos Apoyo RUANA, foto de perfil
Panel admin	Login por código, KPIs, pendientes de validación, pagos, conflictos, chats, campañas de invitación, competencias, reglas
Contactos RUANA	Ciclo iniciado → aceptado → trabajo → cierre / no concretado / disputa
Chat	Mensajes por contacto; vigencia 48 h ; límite actual en código: 30 mensajes totales por conversación
Score	0-100, límite ± 10 /día; estados PRIORITARIO / ESTABLE / EN RIESGO / COMPETENCIA
Reglas de score	Referidos (+3), pago Apoyo (+2), linaje gen1/gen2 (+1), 4 encargos limpios/mes (+3), penalizaciones por contactos abiertos / disputas
Competencia	Si score < umbral, suplente temporal; finalización por vencimiento o admin
Purga mensual	Script + endpoint admin; suspensión temporal según reglas de config
Invitaciones	Por código de aliado, por oficio (<code>RUANA-...</code>), campañas multiuso admin
Referidos / linaje	Árbol, hijos, ruta, backfill desde <code>invitado_por_codigo</code>
Pagos / Apoyo RUANA	Porcentaje desde config (<code>apoyo_pct</code>); comprobantes en Storage; estados de pago; impugnaciones
Storage	Supabase: fotos (<code>ruana-public</code>), comprobantes (<code>ruana-comprobantes</code>)

Área	Qué hay
Motor de evaluación	Filtros tasa respuesta/confirmación/meses sin trabajo → verde/amarillo/rojo + severidad

Páginas HTML legacy (`private-panel.html` , `private-panel-new.html` , `dashboard.html` como redirect) pueden existir pero el flujo principal documentado y servido es `index/invite` → `register` → `aliado` → `admin`.

Flujo general de uso

1. **Entrada** (`/` o `/invite`): el usuario introduce un código de invitación o inicia sesión como aliado con su código.
2. **Validación de invitación**: `GET /api/validar-invitacion?codigo=...`; el frontend guarda contexto en `sessionStorage`.
3. **Registro** (`/register`): `POST /api/aliados/registrar` → login automático → redirección a `/aliado`.
4. **Panel aliado**: sesión por cabecera; carga `GET /api/aliado/datos` y APIs de solicitudes, contactos, chat, pagos.
5. **Admin** (`/admin`): login con código admin; consume APIs bajo `/api/admin/*` y métricas protegidas.

API (resumen)

Base local: `http://127.0.0.1:5000`.

Categorías principales registradas en `app.py`:

- **Páginas**: `/`, `/invite`, `/register`, `/aliado`, `/admin`, `/static/...`
- **Auth**: `/api/aliado/login|sesion|logout`, `/api/admin/validar|logout|me`
- **Aliados**: registro, datos propios, directorio de grupo, foto de perfil, notificaciones
- **Solicitudes**: listar/crear/atender (aliado) y listado/atención admin
- **Invitaciones / campañas**: validar, crear, generar por oficio, campañas admin
- **Contactos / chat / importes / comprobantes / impugnaciones**
- **Métodos de pago** (lectura aliado; escritura admin)
- **Evaluaciones, competencia, purga, reglas**
- **Métricas / health / stats** (muchas rutas de stats requieren admin)
- **Admin**: pendientes, pagos, conflictos, conversaciones, referidos, competencias, etc.

La lista exhaustiva de rutas está en `RUANA/web/app.py` (decoradores `@app.route`). El documento `RUANA/README.md` enumera muchas rutas históricas; **algunas ya no coinciden** con el código (por ejemplo, la ruta antigua `/api/solicitudes/grupo`).

Scripts disponibles

npm (package.json)

Script	Acción
<code>npm run qa:e2e</code>	Playwright
<code>npm run qa:e2e:headed</code>	Playwright con UI
<code>npm run qa:report</code>	Abre reporte HTML en <code>qa-artifacts/playwright-report</code>
<code>npm run verify:supabase</code>	Verifica esquema/buckets Supabase
<code>npm run verify:runtime</code>	Health check (PowerShell)
<code>npm run firebase:login / firebase:deploy</code>	Firebase Hosting
<code>npm run supabase:login / link / push</code>	CLI Supabase
<code>npm run cloudrun:build / cloudrun:deploy</code>	Build/deploy Cloud Run
<code>npm run deploy:hosting / deploy:preview</code>	Hosting / canal preview
<code>npm run gcp:secrets / gcp:allow-dev-wif</code>	Secretos y WIF

Python (RUANA/scripts/)

Script	Acción
<code>seed_aliados.py</code>	Datos semilla
<code>purga_mensual.py</code>	Purga mensual (cron)
<code>verify_supabase.py</code>	Comprobaciones Supabase
<code>test_foto_perfil_e2e.py</code>	E2E de foto de perfil contra despliegue
<code>cron_purga_mensual.txt</code>	Plantilla crontab (incluye una ruta de ejemplo local; hay que adaptarla)

PowerShell (scripts/)

`deploy_cloudrun.ps1`, `deploy_firebase_hosting.ps1`, `set_gcp_secrets.ps1`,
`verify_runtime.ps1`, `allow_dev_branch_wif.ps1`, `ruana_env.ps1`.

Pruebas

Unitarias / integración (pytest)

```
pip install -r RUANA/web/requirements-dev.txt
python -m pytest RUANA/tests -q
```

Cobertura aproximada de `RUANA/tests/` : permisos (Hito 2A/2B), sesiones JWT aliado, chat, pagos/disputas, métodos de pago, storage, foto de perfil, invitaciones/campañas, referidos/linaje, score (reglas 3 y 4), solicitudes, contratos frontend, responsive.

E2E (Playwright)

```
npm ci
npx playwright install --with-deps chromium
npm run qa:e2e
```

Por defecto arranca `python RUANA/web/run.py` con SQLite temporal (sin Supabase). Detalle: `docs/QA_TESTING_PLAN_RUANA.md`.

CI

- `.github/workflows/ruana-qa.yml` — QA manual (pytest + Playwright), dispara con `workflow_dispatch`.
- `.github/workflows/deploy-firebase.yml` — deploy producción en push a `main`.
- `.github/workflows/deploy-firebase-preview.yml` + `trigger-preview-deploy.yml` — preview desde rama `dev`.

Despliegue

Contenedor

Dockerfile:

- Base `python:3.13-slim`
- Instala `RUANA/web/requirements.txt`
- Copia `RUANA/` → `/app/`
- CMD: `gunicorn --bind :${PORT:-8080} --workers 1 --threads 8 --timeout 0 web.app:app`

Firebase Hosting

- Site/proyecto: `ruana-4293f`
- `firebase-public/` está vacío salvo `.gitkeep`
- Rewrite `**` → Cloud Run service `ruana` en `europe-west1`

Cloud Run (producción)

Según workflows/scripts:

- Servicio `ruana`, región `europe-west1`
- Imagen en Artifact Registry `europe-west1-docker.pkg.dev/ruana-4293f/ruana/ruana`
- Secretos típicos: `DATABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_ANON_KEY`, `FLASK_SECRET_KEY`
- Preview: servicio `ruana-preview` (no modifica producción)

URL de hosting documentada en el workflow de producción: `https://ruana-4293f.web.app`.

Pendientes y roadmap

Fuente: ROADMAP.md (fecha de creación 2026-05-22). Estado resumido:

Hito	Estado documentado
1 — Auditoría y base de despliegue	Cerrado documentalmente
2 — Cierre de superficie crítica	Activo (2A/2B avanzados; revisar criterios de salida)
3 — Endurecimiento técnico	Pendiente (secret key obligatorio, CORS, FK SQLite, sesiones persistentes, cron interno, etc.)
4 — Coherencia funcional	Pendiente (motor vs score operativo, nomenclatura de estados, alineación Apoyo RUANA, legacy UI)
5 — Calidad y operaciones	Pendiente (más cobertura, migraciones menos dispersas, logs estructurados)
6 — Despliegue real / beta	Pendiente
7 — MVP público controlado	Pendiente

Observaciones factuales adicionales del código (no inventadas como “bugs cerrados”):

- El orquestador instancia `MetricsCollector` sin inyectar `db` en el flujo por defecto; el collector sin DB devuelve métricas vacías.
- `MotorEvaluacion` calcula `delta_score` pero no aplica ese delta a `aliados.score` en el propio motor.
- Documentación en `RUANA/docs/LOGICA_CHAT_Y_ALERTA.md` aún habla de 5 mensajes/usuario; el código usa **30 mensajes totales**.
- `RUANA/README.md` sigue describiendo SQLite como única fuente de verdad y auth por cookie; el código ya soporta Postgres/Supabase y `X-Ruana-Session-Id`.

Documentación adicional

Documento	Contenido
ROADMAP.md	Prioridades y hitos operativos
RUANA/README.md	Documento de dominio histórico/extendido (contrastar con el código)
RUANA/docs/AUTENTICACION_SESIONES_SEGURAS.md	Diseño de sesiones
RUANA/docs/LOGICA_CHAT_Y_ALERTA.md	Chat y alertas de contactos
RUANA/docs/FLUJO_REGISTRO_ALIADOS_OFICIOS.md	Registro y catálogo de oficios
docs/QA_TESTING_PLAN_RUANA.md	Plan y matriz QA
docs/superpowers/	Specs y planes de hitos concretos

Límites conocidos

- **Producción:** no desplegar con `FLASK_SECRET_KEY` por defecto ni sin Postgres/secretos gestionados.
 - **Sesiones en memoria:** no aptas para múltiples réplicas sin store compartido.
 - **SQLite:** adecuado para desarrollo y QA local; concurrencia alta no es el objetivo de este modo.
 - **Uploads / Storage:** requieren Supabase configurado; sin él fallan rutas de foto/comprobantes.
 - **Páginas y JS legacy:** algunos HTML/JS (`private-panel*` , `dashboard.js` , `referidos-module.js`) no están enlazados en el flujo principal actual; su estado de mantenimiento no se deduce del código más allá de su presencia en el árbol.
 - **Licencia:** no consta en el repositorio un archivo de licencia; el modelo de distribución/uso **no puede deducirse** del código.
-

Licencia

No se ha encontrado un archivo `LICENSE` en el repositorio. El tipo de licencia **no está especificado** en los archivos revisados.